PSP (PYTHON FOR PROBLEM SOLVING) — IA-1 EXAM KIT

Exam: Wednesday 29 October 2026 (per student; no separate date announcement was in the Slack dumps).

Generated: 2026-10-28 · **Batch:** S1 B'26-CS B (PSP B)

Sources: portal export `PSP-questions.md` (100 assignments, 9 quizzes, 14 lecture windows, Slack sheets, 47-question workbook — captured 23 Oct), live portal pages captured 28 Oct (47/67 assignments solved, 3/9 quizzes), Slack dumps `academic-announcement-26.md` / `psp-26.md` (to 23 Oct), syllabus announcement 14 Oct.

Everything below is from those sources or standard Python — nothing invented.

PART 0 — EXAM CARD (read first, 3 minutes)

Syllabus — VERBATIM from the 14 Oct announcement ("Syllabus for IA-1 PSP")

#	Announcement line
L1	Lecture - 1 Outline (Flowcharts)
L2	Lecture - 2 Outline (Variable, Datatype, Typecasting)
L3	Lecture - 3 Outline (Input, Arithmetic operator)
L4	Lecture - 4 Outline (if else)
L5	Lecture - 5 Outline (if else ladder)
L6	Lecture - 6 Outline (Nested if else)
L7	Lecture - 7 Outline (Introduction to Function)
L8	Lecture - 8 Outline (Function with if else)
L9	Lecture - 9 Outline (Logical operators, Function with nested if-else)
L10	Lecture - 10 Outline (Function with recursive thinking)
L11	Lecture - 11 Outline (For loop)
L12	Lecture - 12 Outline (While loop)

What is confirmed / not confirmed

- **Confirmed:** syllabus = L1–L12 above; your real state: 47/67 assignments solved, 3/9 quizzes, Contest 2 partially solved.
- **NOT in any dump (through 23 Oct):** an announced paper pattern, marks, duration, or venue for PSP IA-1. Students asked for the "finalized IA1 syllabus and paper pattern for all 5 subjects" on 17 Oct; only syllabuses were posted.
- Sibling facts (real, from the same channel): Math-1 IA-1 was announced as **a written paper**; AP IA-1 ran as a **S-VYASA written paper** (5×3M + 3×5M choice, 2 h, no calculator); PSP and other contests ran **on Athena, 9:00–11:00 AM**.

Protocol — be ready for BOTH shapes of paper

If it is a written paper (most likely):

1. Write full programs, not pseudocode: correct indentation, `:` after `if` / `for` / `while` / `def`, `print(...)` with commas where asked.

2. Read-aloud rule: before writing, say the exact output string in quotes. Spelling/case is marks (`"Yes"` not `"YES"` unless the statement says YES).

3. Order: do L11/L12 loop-writing and L4–L6 conditionals first (they are your 100 recent assignments), functions last (they take longest to write cleanly).

4. Every loop question: run the sample input on paper once, line by line, before you move on.

If it is an MCQ/online test on Athena:

1. Trace-before-answer: write the variable table for loops (`i`, `x`) — that is how every quiz question in this course was written.

2. Check whether the paper states negative marking. If it does not, attempt everything; if it does, still attempt anything you can half-solve — but follow the printed rule exactly.

3. Never leave a blank on a written section.

Desk order (both shapes): 60-second morning card (Part 6) → loop patterns (Part 2, types 9–13) → condition ladder patterns (types 4–6) → solutions you have not re-read (Part 3).

PART 1 — CONCEPT COURSE, L1 → L12

L1 — Flowcharts

- **What:** a diagram of the program: oval = start/end, parallelogram = input/output, rectangle = process, diamond = decision, arrows = flow.
- **Exam form:** "draw a flowchart for…" or "what shape is the decision box?" — answer: **diamond** for decisions, **rectangle** for assignments, **parallelogram** for input/output, **oval** for terminal.
- **Gotcha:** arrows must show a loop going back, and every diamond has exactly two labelled exits (Yes/No or True/False).
- **Tested in:** PSP Contest 1 syllabus announcement (Flowchart was question 1 of the syllabus list, 1 Sep).

L2 — Variables, datatypes, typecasting

```
x = 10            # int

pi = 3.14         # float

name = "Arush"    # str

flag = True       # bool

n = int("42")     # str -> int (typecast)

f = float("3.5")  # str -> float

s = str(99)       # int -> str

type(x)           # <class 'int'>
```

- **Gotchas:** `input()` ALWAYS returns a string — `n = input()` then `n + 1` crashes (`str + int`). Integer division `//`, remainder `%`. `int("3.7")` crashes — use `float` first.

- **Tested in:** quizzes "Print function…Python Variable…Type() function" (2 Sep), assignments Swap Two Variables, Find The Square, Integer Type.

L3 — Input and arithmetic operators

```
a = int(input())

b = int(input())

print(a + b, a - b, a * b, a / b, a // b, a % b, a ** 2)
```

- **Gotchas:** precedence is `**` > `* / // %` > `+ -`. `/` always float (`7/2 = 3.5`), `//` floors. Two inputs on ONE line: `l, r = map(int, input().split())`.
- **Tested in:** Swap Two Variables (60/60), Rectangle Area, Energy Consumption, Celsius to Fahrenheit, Convert Minute to Hour, Compound Interest, Contest-2 "Find the remainder".

L4 — if–else

```
if condition:

    # runs when True

else:

    # runs when False
```

- **Gotchas:** the `:` is required; indentation defines the block (4 spaces); `==` compares, `=` assigns; condition can be a comparison directly (`if n % 2 == 0:`).
- **Tested in:** Even or Odd Checker, Who is Greater?, Weather Report, AM_PM, Identify Square or Rectangle, Profit or Loss; quiz "Conditional Statements, Python if-else" (9 Sep, you scored 8/8).

L5 — if–elif–else ladder

```
if score >= 90:

    grade = "A+"

elif score >= 75:

    grade = "A"

elif score >= 60:

    grade = "B"

else:

    grade = "C"
```

- **Gotchas:** only ONE branch runs; `elif` not `else if`; order matters — put the strictest/highest condition first; last branch can be bare `else`.
- **Tested in:** Grade Calculator, The Number Detective, FizzBuzz II, Income Tax, Electricity Bill, Triangle Type, Library fine, Salary, BMI; Contest-2 "The Gatekeeper" and "Function Based Calculator"; quiz "If-elif-else ladder" (21 Sep, 0/8 — unattempted).

L6 — Nested if–else

```
if a + b > c:

    if a + c > b:

        if b + c > a:

            print("Triangle valid")

        else:

            print("Invalid")

else:

    print("Invalid")
```

- **Gotchas:** every inner `if` needs its own `:` and indentation level; `and` / `or` (L9) can flatten nesting — but the syllabus wants you to be able to read BOTH forms.
- **Tested in:** Can It Be a Triangle?, Leap Year Checker, Roller Coaster, Accommodation Decision, Find the Smallest/Biggest; quizzes "Python Function, Nested if-else" (23 Sep, 0/14).

L7 — Introduction to functions

```
def average(a, b, c):

    return (a + b + c) / 3



print(average(80, 90, 70))
```

- **What:** `def name(params): body` — define once, call anywhere after it. `return` sends a value out; `print` only shows it.
- **Gotchas:** the function does NOTHING until called; `return` with no value gives `None`; a function without `return` cannot be used in an expression.
- **Tested in:** Calculate Percentage Using Function, Percentage-1/-2, Average of Three; workbook Units 1–7.

L8 — Function with if–else

```
def get_grade(m1, m2, m3):

    if m1 < 40 or m2 < 40 or m3 < 40:

        return "Fail"

    pct = (m1 + m2 + m3) / 300 * 100

    if pct >= 90:

        return "A+"

    # ...
```

- **Gotchas:** `return` exits immediately — no code after it runs; decide return TYPE (string vs number) from the statement, not from how you tested.
- **Tested in:** Subject Grades, Smart Conditional Calculator, Grade Report (Contest 2 — in Part 3); quiz "Python Function…Arguments and parameter, Return" (28 Sep, 8/8 — your best score).

L9 — Logical operators + functions with nested if–else

```
if a > b and b > c:      # both must be True

if x == 1 or x == 2:     # either True

if not flag:             # flips True/False
```

- **Gotchas:** `and` / `or` / `not` (words — never `&&` / `||`); short-circuit: `x != 0 and 10/x > 1` never divides by zero when `x == 0`; precedence: `not` > `and` > `or`.
- **Tested in:** Voter Eligibility, Greater or Equal, NST Library, Museum Booking, Greater of Four; Contest-2 Gatekeeper; assignment sets due 5 Oct (logical operators).

L10 — Function with recursive thinking

```
def factorial(n):

    if n <= 1:        # BASE CASE — without it: infinite recursion / crash

        return 1

    return n * factorial(n - 1)



def sum_to(n):

    if n <= 0:

        return 0

    return n + sum_to(n - 1)
```

- **What:** a function that calls itself with a SMALLER input, until a base case stops it.
- **Gotchas:** every recursion needs a base case; each call multiplies stack frames (deep recursion hits the limit); trace with a call tree on paper.
- **Tested in:** workbook Unit 12 (Recursion Made Easy), sum_to / sign / factorial patterns in Slack sheets; "Factorial-Loops" and "Is It Prime?" carry Recursion in their topic tags (Part 3 has loop versions — both forms examinable).

L11 — For loop

```
for i in range(1, n + 1):   # 1..n inclusive

    print(i * i)

for i in range(n):          # 0..n-1

    ...

for ch in "abc":            # iterate a string

    ...
```

- **Gotchas:** `range(a, b)` stops at `b-1`; `range(n)` starts at 0; accumulating needs `total = 0` BEFORE the loop (a `total = 0` inside prints the last value only); print-on-same-line vs newline: `print(x, end="")`.
- **Tested in:** Sum 1 to N, Multiplication Table, Print all divisors, Multiples of 3 and 5, Squares I/II, Count Divisible by 3 or 5, Fibonacci, Odd Factors, First Number Divisibly by 7 (Part 3).

L12 — While loop

```
n = int(input())

while n > 0:

    digit = n % 10      # take last digit

    n //= 10            # drop last digit
```

- **What:** repeats while the condition stays True — use when you don't know how many iterations (digit counting, reversing, searching).
- **Gotchas:** you must CHANGE something the condition depends on, or it loops forever; `break` exits the loop, `continue` skips to the next check; `while True` + `break` is legal.
- **Tested in:** Count the Digits, Reverse a Number, Digit Sum, First+Last Digit, Even/Odd digit counts, Largest/Smallest Digit, Armstrong, Palindrome number, All Digits Same, Check if Rare, Perfect Number (all in Part 3); quizzes "While loop" (12 Oct, 0/10), "Control flow, Break, Continue" (16 Oct, 0/8).

PART 2 — PATTERN LIBRARY (the 13 archetypes every question in this course is made of)

Copy the skeleton, fill the blanks. Question names are the real ones from your portal.

TYPE 1 — Print pattern (L1/print) — *Mirror Triangle, Inverted Triangle, Numbers Triangle, Hollow Square, No Time to Newline*

```
n = 5

for i in range(1, n + 1):

    print("*" * i)                 # left-aligned triangle

for i in range(n, 0, -1):

    print("*" * i)                 # inverted

for i in range(1, n + 1):

    print(" " * (n - i) + "*" * i)  # mirror (right-aligned)
```

TYPE 2 — Input + conversion formula (L3) — *Celsius→Fahrenheit, Rectangle Area, Energy, Minute→Hour, Compound Interest, BMI*

```
c = float(input())

print(c * 9 / 5 + 32)          # exact string as sample says

h, m = divmod(total_min, 60)   # or: h = total_min // 60; m = total_min % 60
```

TYPE 3 — Two-branch classify (L4) — *Even/Odd, Profit/Loss, Weather, AM/PM, Square/Rectangle*

```
if n % 2 == 0:

    print("Even")

else:

    print("Odd")
```

TYPE 4 — Ladder with thresholds (L5) — *Grade Calculator, Income Tax, Electricity Bill, BMI, Library Fine, Salary*

```
if x < a:        ...

elif x < b:      ...   # always ascending thresholds

elif x < c:      ...

else:            ...
```

Rule: write the boundaries in strictly increasing order; test the boundary value itself (exactly 75, exactly 40).

TYPE 5 — Nested validation (L6) — *Can It Be a Triangle, Leap Year, Roller Coaster, Accommodation*

```
if a + b > c and a + c > b and b + c > a:   # or 3 nested ifs (syllabus L6 style)
    print("YES")
else:
    print("NO")


# leap year: divisible by 4, but century years only if divisible by 400
if year % 400 == 0 or (year % 4 == 0 and year % 100 != 0):
    print("Leap")
```

TYPE 6 — Function + decision (L8) — *Subject Grades, Smart Calculator, Grade Report*

```
def solve(...):
    if <fail condition>:
        return <fail value>
    # compute
    return <answer>
# no input()/print() in the function unless the statement says so
```

TYPE 7 — Logical eligibility (L9) — *Voter Eligibility, Gatekeeper, NST Library, Museum Booking*

```
if u == "admin" and p == "password123":
    print("Login successful")
elif u == "admin" or p == "password123":   # exactly one matches
    print("Check username or password")
else:
    print("Invalid credentials")
```

TYPE 8 — Recursion (L10) — *sum_to, factorial, sign, recursion in workbook Unit 12*

```
def f(n):

    if n < 2:            # base case FIRST

        return ...

    return ... f(n - 1)
```

TYPE 9 — For-loop accumulate (L11) — *Sum 1..N, divisors, multiples, table, squares, Fibonacci*

```
total = 0                    # BEFORE the loop — classic bug if inside

for i in range(1, n + 1):

    if condition(i):

        total += i

print(total)
```

TYPE 10 — For-loop build a string/list, then print (L11) — *Squares II, Count Divisible by 3 or 5, Fibonacci, Odd Factors*

```
out = []

for i in range(1, n + 1):

    if cond(i):

        out.append(str(i))

print(" ".join(out))          # one line, space-separated
```

TYPE 11 — While digit-extraction (L12) — *Count digits, Reverse, Digit sum, First+last, Even/odd counts, Largest digit, Add 1 to each digit, Armstrong*

```
n = int(input())

while n > 0:

    d = n % 10        # last digit

    ...               # use d

    n //= 10          # drop it
```

One template, 8 questions. Variants: count (`c += 1`), sum (`s += d`), reverse (`r = r * 10 + d`), max (`if d > m: m = d`), first digit (strip until `n < 10`), add-1 (`nd = (d + 1) % 10` — 9 becomes 0).

TYPE 12 — While + break early-exit (L12, L9) — *First Number Divisibly by 7, Check if All Digits are Same, Check if Rare*

```
found = None

for i in range(l, l + 7):        # at most 7 candidates — even if R is 10^18

    if i % 7 == 0:

        found = i

        break

print(found if found else "NOT FOUND")
```

TYPE 13 — Number theory with loop (L11/L12) — *Is It Prime?, Perfect Number, Palindrome number*

```
# prime: check only up to sqrt(n)

i = 2

while i * i <= n:

    if n % i == 0:

        break

    i += 1

# palindrome: reverse and compare, or compare str to str[::-1] if strings allowed
```

PART 3 — SOLVED COUNTER (your 23 unsolved questions, all solved and sample-verified)

53/53 sample checks pass (every sample case in every statement was run — verified 28 Oct). Statements quoted verbatim where it matters (exact output strings, constraints).

C-1. The Gatekeeper — Contest 2, medium, 0/80 (if-elif-else ladder)

> "The correct username is `admin`. The correct password is `password123`." Print `Login successful` if both correct; `Check username or password` if exactly one correct; `Invalid credentials` if both wrong.

```
u = input().strip()

p = input().strip()

ok_u = (u == "admin")

ok_p = (p == "password123")

if ok_u and ok_p:

    print("Login successful")

elif ok_u or ok_p:

    print("Check username or password")

else:

    print("Invalid credentials")
```

Samples: `admin/password123 → Login successful`; `admin/wrongpass → Check username or password`. *(Note: the statement's em-dashes are punctuation — samples show plain strings.)*

C-2. Newton and Rounding — Contest 2, hard, 0/120 (arithmetic operators)

> Round n to nearest multiple of 10. Ends with 0 → same. Last digit 1–4 → lower. 6–9 → higher. **Last digit 5 → always LOWER.**

```
n = int(input())

d = n % 10

if d <= 5:

    print(n - d)        # 0,1,2,3,4,5 all go DOWN (5 is the trap)

else:

    print(n + (10 - d))
```

Samples: `5 → 0` · `113 → 110` · `2359 → 2360`. Also checked: `0 → 0`, `10 → 10`.

C-3. Grade Report — Contest 2, medium, 0/80 (functions)

> Complete `def get_grade(mark1, mark2, mark3)`. Any subject < 40 → return `Fail`. Else percentage of 300: ≥90 → `A+`, 75–<90 → `A`, 60–<75 → `B`, 40–<60 → `C`. Return only — no input/print.

```
def get_grade(mark1, mark2, mark3):

    if mark1 < 40 or mark2 < 40 or mark3 < 40:

        return "Fail"

    pct = (mark1 + mark2 + mark3) / 300 * 100

    if pct >= 90:

        return "A+"

    if pct >= 75:

        return "A"

    if pct >= 60:

        return "B"

    return "C"
```

Samples: `95,92,88 → A+` (91.7%) · `80,90,70 → A` (80%) · `75,35,80 → Fail`.

C-4. Count Numbers Divisible by 3 or 5 — 0/20 (for loop)

> Print all i in 1..N divisible by 3 or 5 **but not both** (XOR). `3≤N≤100`. Sample: `15 → 3 5 6 9 10 12`.

```
n = int(input())

out = []

for i in range(1, n + 1):

    if (i % 3 == 0) ^ (i % 5 == 0):     # XOR: exactly one of them

        out.append(str(i))

print(" ".join(out))
```

C-5. Squares from 1 to N (Part I) — 0/20 (for loop)

> Print squares $1^2..N^2$, **one per line**. `1≤N≤20`.

```
n = int(input())

for i in range(1, n + 1):

    print(i * i)
```

C-6. Squares from 1 to N (Part II) — 0/20 (for loop)

> Same squares **on one line, space-separated**. Sample: `5 → 1 4 9 16 25`.

```
n = int(input())

print(" ".join(str(i * i) for i in range(1, n + 1)))
```

(If generators are banned in your environment: build a list with `append` first — TYPE 10.)

C-7. Factorial-Loops — 0/20 (loops/recursion)

> N! for `0≤N≤20`. `0! = 1`.

```
n = int(input())

f = 1

for i in range(2, n + 1):

    f *= i

print(f)
```

Samples: `5 → 120`, `10 → 3628800`, `0 → 1`.

C-8. Check if a Number is a Palindrome — 0/20 (while/recursion)

> "reads same backwards and forwards". Sample: `54145 → Yes`, `12345 → No`.

```
n = int(input())

s = str(n)

print("Yes" if s == s[::-1] else "No")
```

Loop version (if strings blocked, as in "Count the Digits"): reverse the number with TYPE 11 and compare `r == original`.

C-9. Digit Sum using Loop — 0/20 (function only)

> `sumOfDigits(n)` must **return** the sum. `28393 → 25`.

```
def sumOfDigits(n):

    s = 0

    while n > 0:

        s += n % 10

        n //= 10

    return s
```

C-10. Count the Digits — 0/20 (while)

> **Blocked in backend:** string literals, list literals, `str()` / `list()` conversions, indexing/iteration/string methods on `input()`. So pure arithmetic. `28232 → 5`.

```
n = int(input())

c = 0

while n > 0:

    c += 1

    n //= 10

print(c)
```

C-11. Reverse a Number — 0/20 (while)

> `536446 → 644635`; `100 → 1` (no trailing zeros kept).

```
n = int(input())

r = 0

while n > 0:

    r = r * 10 + n % 10

    n //= 10

print(r)
```

C-12. Sum of First and Last Digits — 0/20 (while)

> `14368 → 9` (1 + 8).

```
n = int(input())

last = n % 10

while n >= 10:

    n //= 10          # n is now the first digit

print(n + last)
```

Edge (no sample given): single digit 7 → first = last = 7 → literal reading gives **14**. Keep the literal formula; if the judge disagrees on that one edge, it is the only unverifiable case in this file.

C-13. Find the Count of Even and Odd Digits — 0/20 (while)

> `295892 → 3 3` (even count first, then odd, space-separated).

```
n = int(input())

e = o = 0

while n > 0:

    d = n % 10

    if d % 2 == 0:

        e += 1

    else:

        o += 1

    n //= 10

print(e, o)
```

C-14. Find the Largest Digit in a Number — 0/40 (while)

> `2839 → 9`. **`0≤N≤10^9` — N can be 0** (sample-free edge: print 0).

```
n = int(input())

m = 0

while n > 0:

    d = n % 10

    if d > m:

        m = d

    n //= 10

print(m)
```

(Works for N = 0: loop never runs, prints 0.)

C-15. Is It Prime? — 0/20 (loops)

> Print exactly `YES` / `NO`. `1` is NOT prime. `7 → YES`, `10 → NO`.

```
n = int(input())

if n < 2:

    print("NO")

else:

    i = 2

    prime = True

    while i * i <= n:

        if n % i == 0:

            prime = False

            break

        i += 1

    print("YES" if prime else "NO")
```

C-16. Add 1 to Each Digit — 0/20 (loops)

> "adding 1 to digit 9 makes it 0". `765392 → 876403`.

```
n = int(input())

res = 0

mul = 1

while n > 0:

    d = (n % 10 + 1) % 10     # 9 -> 0, others -> +1

    res += d * mul

    mul *= 10

    n //= 10

print(res)
```

Edge: `9 → 0` (leading zeros vanish when printed as an integer — matches "print an integer").

C-17. Fibonacci Series of a given input — 0/20 (loops)

> First N terms, space-separated, starting **0 1 1 2 3**. `5 → 0 1 1 2 3`. `1≤N≤20`.

```
n = int(input())

a, b = 0, 1

out = []

for _ in range(n):

    out.append(str(a))

    a, b = b, a + b

print(" ".join(out))
```

C-18. Find Odd Factors — 0/20 (for loop)

> All odd factors of N in increasing order. `21 → 1 3 7 21`. `1≤N≤10^5`.

```
n = int(input())

out = []

for i in range(1, n + 1):

    if n % i == 0 and i % 2 == 1:

        out.append(str(i))

print(" ".join(out))
```

(1 is always included — the sample confirms it.)

C-19. Check if Even Sum is Divisible by 2 — 0/20 (while/loop)

> Sum all EVEN numbers in [L, R], print `Yes` if that sum % 2 == 0 else `No`. **`1≤L≤R≤10^18` — you CANNOT loop the range.**

```
l, r = map(int, input().split())

first = l if l % 2 == 0 else l + 1

last  = r if r % 2 == 0 else r - 1

if first > last:

    s = 0

else:

    k = (last - first) // 2 + 1          # count of evens

    s = k * (first + last) // 2          # arithmetic-series sum

print("Yes" if s % 2 == 0 else "No")
```

Sample: `23 29 → Yes` (24+26+28 = 78). Also checked: no evens in range → sum 0 → `Yes`; L=R=10^18 → `Yes`.

C-20. Check for Perfect Number — 0/20 (while)

> Sum of PROPER divisors (excluding N) equals N. `6 → True` (1+2+3). Print `True` / `False`.

```
n = int(input())

s = 0

i = 1

while i * i <= n:

    if n % i == 0:

        if i != n:

            s += i

        other = n // i

        if other != i and other != n:

            s += other

    i += 1

print("True" if s == n else "False")
```

Checked: `6 → True`, `28 → True`, `10 → False`.

C-21. First Number Divisibly by 7 — 0/20 (for + break)

> Smallest multiple of 7 in [L, R], else `NOT FOUND`. **`R` up to 10^18 — but only 7 candidates can matter.**

```
l, r = map(int, input().split())

ans = "NOT FOUND"

for i in range(l, min(r, l + 6) + 1):

    if i % 7 == 0:

        ans = str(i)

        break

print(ans)
```

Samples: `23 29 → 28` · `23 27 → NOT FOUND` · exact `7 7 → 7` · huge range ending 10^18 → `999999999999999999` (10^18 ≡ 1 mod 7).

C-22. Check if All Digits are Same — 0/40 (while + break)

> `77777 → Yes`, `22888 → No`.

```
n = int(input())

first = n % 10

ok = True

while n > 0:

    if n % 10 != first:

        ok = False

        break

    n //= 10

print("Yes" if ok else "No")
```

C-23. Check if Rare — 0/40 (while + break)

> "rare if all of its digits are divisible by X". Input `N X`. `28 2 → True`.

```
n, x = map(int, input().split())

ok = True

while n > 0:

    if (n % 10) % x != 0:

        ok = False

        break

    n //= 10

print("True" if ok else "False")
```

Checked: `28 3 → False`.

PART 4 — DRILL INDEX (what exists, what is done, what is left)

4.1 Your 9 quizzes (portal state, captured 28 Oct) — 3/9 solved

| Quiz | Window / date | Score | Revise with |

|---|---|---|---|

| Print function, Comments, Variable, Primitive, `type()` | 2 Sep | **0/12** | L2 · assignments: Swap Two Variables, Find The Square, Integer Type |

| Input, Arithmetic operators, precedence | 7 Sep | 2/26 | L3 · Rectangle Area, Energy, C→F, Minute→Hour |

| Conditional Statements, if-else | 9 Sep | **8/8 ✓** | L4 — your strongest area |

| If-elif-else ladder | 21 Sep | **0/8** | L5 · Grade Calculator, Income Tax, Electricity Bill, Triangle Type |

| Python Function, Nested if-else | 23 Sep | **0/14** | L6–L7 · Can It Be a Triangle?, Leap Year, Subject Grades |

| Function, Nested if-else, Arguments, Return | 28 Sep | **8/8 ✓** | L8 — keep this level |

| Iteration, Recursion, Range | 12 Oct | **0/14** | L10–L11 · TYPE 8 + TYPE 9/10 patterns |

| While loop | 12 Oct | **0/10** | L12 · all of Part 3 C-9…C-14 |

| Control flow, Loops, Break, Continue | 16 Oct | **0/8** | L11–L12 · TYPE 12 (Div7, All Same, Rare) |

Question text for the unattempted quizzes is NOT in any file — the portal only shows questions after starting a test (forbidden by the rules of this project). Revise by pattern, not by past paper.

4.2 Assignment sets solved (the 47 you have done — quick re-read list by date)

- **8 Sep (L2):** Print Mirror Right Angle Triangle · Print Inverted Triangle · Print Numbers Right-Angled Triangle · Print Hollow Square · No Time to Newline *(all 10/20)*
- **12 Sep (L3):** Swap Two Variables (60/60) · Find The Square · Rectangle Area Calculator · Calculate and Print Age · Energy Consumption · C→F Conversion · Minute→Hour Format
- **14 Sep (L4):** Average of Three · Compound Interest –1 · Even or Odd · Integer Type · Who is Greater? · Weather Report · AM_PM · Identify Square or Rectangle
- **21 Sep (L4–L5):** Absolute Difference · Profit or Loss · Voter Eligibility II · Grade Calculator · The Number Detective · Greater or Equal
- **26 Sep (L5):** FizzBuzz II · Income Tax (40/40) · Electricity Bill (partial 30/60) · Triangle Type · All Three Equal · Find the Biggest! · Library fine · Calculate salary · Check Profit or Loss · BMI-1
- **28 Sep (L6):** Can It Be a Triangle? · Leap Year (40/40) · Roller Coaster · Accommodation Decision · Tax by Age & Income · Find the Smallest!
- **3–5 Oct (L7–L9):** Percentage Using Function · Percentage-1/-2 · Greater/Smaller ×2 · Greater of Three · Subject Grades · Bridge Between Villages · Electricity Bill-2 · Smart Conditional Calculator · C→F-2 · Leap Year or Not · Great Dragon · Function Based Calculator · NST Library (60/60) · Movie Mania · Will Ramandeep Sir Fly –2 · Bus Route · Museum Booking · Greater of Four
- **9 Oct (Contest 2):** Find the remainder (40/40) · The Frame (40/40) · Function Based Calculator (80/80) · Calculate Final Price (40/80) — **backlog: The Gatekeeper, Newton and Rounding, Grade Report → Part 3**
- **10–19 Oct (L10–L12):** Palindrome with Digit Validation · NewForces Ratings (60/60) · Sum in Range L..R · Multiplication Table · Print divisors · Multiples of 3 and 5 · Find The Even Numbers · Sum 1 to N · Sum of Even/Odd Digits (40/40) · Smallest Digit (40/40) · Armstrong II (40/40)
- **Not yet done (→ Part 3 has all):** Count Divisible 3 or 5 · Squares I & II · Factorial-Loops · Palindrome number · Digit Sum fn · Count the Digits · Reverse · First+Last · Even/Odd counts · Largest Digit · Is It Prime? · Add 1 to Each Digit · Fibonacci · Odd Factors · Even Sum %2 · Perfect Number · Div7 · All Digits Same · Check if Rare

4.3 Workbook — "Python Functions: Practice Workbook" (Slack, 19 Oct; 47 questions, 150 marks, 13 units)

Units: 1 What is a function (5) · 2 Define & call (10) · 3 `return` (10) · 4 `print` (5) · 5 print vs return (15) · 6 Take a value out (10) · 7 Send values in (10) · 8 Decisions/loops inside functions (15) · 9 Functions calling functions + scope (10) · 10 Functions working together (15) · 11 Function reference (15) · 12 Recursion made easy (15) · 13 Final challenge (15).

Full text of all 47 questions is in `export/PSP–questions.md` (lines ~8010–8630) and its PDF. If you do only one thing from this list: Units 5, 8, 12 (print-vs-return, loops-in-functions, recursion) — they map exactly to L7–L10.

4.4 Slack standing directives (real, from #psp-26)

- **2 questions/day minimum** from the two Codeforces sheets (If-Else & Math, 63 problems; Loops, 34 problems) — make your own copy (File → Make a copy) before ticking.
- **12-problem Codeforces code list** shared 22 Oct; **Ashwin's answer key posted 27 Oct** in-channel: `977 A, 978 B, 988 A, 996 A, 731 A, 832 A, 1800 A, 910 A, 115 A, 977 B, 1113 A, 948 A` (A/B suffix = which problem of each pair).

- Workbook PDF link is in the export's sources section.

4.5 What the exam can and cannot draw from

| Source | In-file? | Count |

|---|---|---|

| Assignment questions (full text + status) | yes | 100 (47 solved by you) |

| Contest 2 full text | yes | 7 |

| In-class lecture questions (per lecture) | yes | 14 windows |

| Quiz schedules + your scores | yes | 9 |

| Quiz question text (unattempted) | **no** — needs Start Test (forbidden) | 4 quizzes |

| Quiz-contest MCQs (PSP Contest 1, Wifi Demo, Mock Test) | **no** — same reason | 4 |

| Workbook | yes, full text | 47 |

| Codeforces sheets | titles + ratings only, no statements | 97 + 12 |

PART 5 — GAPS (honest)

1. **Paper pattern, marks, duration, venue for PSP IA-1: not announced** anywhere in the dumps through 23 Oct, and Slack could not be re-harvested tonight (session expired — QR re-login needed). If a classmate has a newer screenshot of the pattern, trust it over this file's assumptions.

2. **Quiz question text for the 4 unattempted quizzes does not exist in any capture** — starting the test to read them would consume an attempt, which this project never does.

3. **Single-digit edge in C-12 (First+Last digit)** has no sample on the portal — literal answer 14, flagged above.

4. **Exact sample output formatting** (trailing spaces, blank lines) was normalised to whitespace-equal comparison during verification; when in doubt on the paper, copy the sample's spacing character-for-character.

5. Physically absent data cannot be manufactured: no past IA-1 PSP paper exists in any source (IA-1 has not happened yet).

PART 6 — 60-SECOND MORNING CARD (screenshot this)

1. **`:` after `if` / `elif` / `else` / `for` / `while` / `def`. 4-space indent.**

2. **`input()` is a string** → `int(input())` unless told otherwise; two values: `a, b = map(int, input().split())`.

3. **Digit loop = one template:** `while n > 0: d = n % 10; n //= 10` → count/sum/reverse/max/first/odd-even/add-1 all from it.

4. **Accumulator starts BEFORE the loop** (`total = 0` outside).

5. **`range(1, n+1)` is 1..n; `range(n)` is 0..n-1; `range(n, 0, -1)` counts down.**

6. **Print exact strings:** `Yes/No`, `True/False`, `YES/NO` — copy from the statement, never invent case.

7. **Prime:** loop to `i*i <= n`; **1 is not prime**; **0! = 1**.

8. **Round-half-down:** last digit 5 goes DOWN (Newton and Rounding).

9. **Function-only questions:** write `def` + `return`, no `input()` / `print()` unless asked.

10. **Huge ranges (10^18):** never loop them — arithmetic formula (even-sum) or ≤7 candidates (div-by-7).

11. **Recursion:** base case FIRST, then `return … f(n−1)`.

12. **break** exits, **continue** skips — diamond flowchart = decision, parallelogram = input/output.

Sources: `export/PSP-questions.md` (23 Oct portal export), live portal pages captured 28 Oct (PSP B: 47/67 assignments, 3/9 quizzes), `materials/slack-questions/academic-announcement-26.md` (syllabus, 14 Oct), `psp-26.md`, `general-announcements-26.md`, `student-progression-batch-b.md`. All 23 solutions in Part 3 were executed against their portal sample cases before publication: 53/53 pass.